

COLLEGE MANAGEMENT SYSTEM

README / Project Documentation

Project Type	CSE / Python Mini Project
Registration No.	26MIM10209
Language	Python 3
Programming Concepts	Functions, Lists, Conditional Statements, Loops, Input/Output

A simple console-based College Management System developed in Python to manage student, faculty, course, attendance, examination, fee, and timetable records.

1. Project Overview

The College Management System is a menu-driven Python application designed to demonstrate basic programming concepts through a practical college-management use case. The program allows users to add and view records for different areas of a college. All records are maintained in Python lists while the program is running.

2. Objectives

- Create a simple and user-friendly college management application.
- Manage basic student and faculty information.
- Store course and teaching information.
- Calculate and display student attendance percentages.
- Calculate grades from examination marks.
- Track fee payment, balance, and payment status.
- Maintain a basic class timetable.
- Demonstrate Python functions, lists, loops, conditions, and user input.

3. Main Features / Modules

Module	Functions	Purpose
Student Management	Add Student / View Students	Stores name, registration number, branch and year.
Faculty Management	Add Faculty / View Faculty	Stores faculty name, ID and department.
Course Management	Add Course / View Courses	Stores course code, name and faculty.
Attendance Management	Add Attendance / View Attendance	Calculates attendance percentage.
Examination Management	Add Marks / View Marks	Stores marks and automatically assigns a grade.
Fees Management	Add Fee / View Fees	Calculates balance and payment status.
Timetable Management	Add Class / View Timetable	Stores day, time, subject and room.

4. Technologies Used

- Programming Language: Python 3
- Interface: Command-line / Console
- Data Storage: In-memory Python lists
- External Libraries: None
- Programming Concepts: Functions, lists, if/elif/else, while loop, input(), type conversion and arithmetic operations.

5. Data Structures Used

The application uses separate lists for each category: students, faculty, courses, attendance, marks, fees, and timetable. Each record is stored as a list inside the corresponding main list.

List	Example Record Structure
students	[name, registration number, branch, year]
faculty	[name, faculty ID, department]
courses	[course code, course name, faculty]
attendance	[student name, total classes, attended classes, percentage]
marks	[student name, subject, score, grade]
fees	[student name, total fee, paid amount, balance, status]
timetable	[day, time, subject, room]

6. Grade Calculation

The examination module assigns grades using the following conditions:

Marks	Grade
90 and above	A+
75–89.99	A
60–74.99	B
40–59.99	C
Below 40	Fail

7. Attendance Calculation

Attendance percentage is calculated using: **Attendance % = (Classes Attended / Total Classes) × 100**. The result is displayed rounded to two decimal places.

8. Fee Calculation

The fee module calculates the remaining balance as: **Balance = Total Fee – Amount Paid**. If the balance is zero or negative, the status is shown as Paid; otherwise it is shown as Pending.

9. Program Flow

1. Start the program.
2. Display the main menu.
3. Select one of the seven management modules or Exit.
4. Choose Add or View inside the selected module.
5. Enter the required information when adding a record.
6. The record is stored in the appropriate list.
7. The program returns to the main menu.
8. Select Exit to terminate the application.

10. How to Run the Project

1. Install Python 3 on the computer.

2. Save the program in a file, for example: college_management.py
3. Open Command Prompt / Terminal in the folder containing the file.
4. Run the program using: **python college_management.py**
5. Use the numbered menu to add or view records.

11. Sample Main Menu

1. Student Management
2. Faculty Management
3. Course Management
4. Attendance Management
5. Examination Management
6. Fees Management
7. Timetable Management
8. Exit

12. Advantages

- Simple and easy-to-understand console interface.
- Separate functions make the program modular.
- Covers multiple common college-management activities.
- Demonstrates practical use of basic Python concepts.
- Automatic calculations are provided for attendance, grades and fees.

13. Limitations

- Data is stored only in memory and is lost when the program exits.
- There is no database or permanent file storage.
- There is no login or authentication system.
- Records are identified mainly through names rather than unique database IDs.
- Input validation is limited; invalid numeric input may cause an error.

14. Future Enhancements

- Add database support using MySQL or SQLite.
- Add update and delete operations.
- Add student search by registration number.
- Add login and role-based access for administrator, faculty and student.
- Add file export such as CSV or PDF reports.
- Add a graphical user interface using Tkinter or another Python GUI framework.
- Add stronger input validation and error handling.

15. Conclusion

The College Management System is a beginner-friendly Python project that combines multiple basic programming concepts into one practical application. It provides a foundation that can later be expanded into a database-driven or graphical college management system.

Project Registration No.: 26MIM10209